

Submission Worksheet

CLICK TO GRADE

<https://learn.ethereallab.app/assignment/IT202-007-F2024/it202-api-project-milestone-2-2024-m24/grade/ass89>

Course: IT202-007-F2024
Assignment: [IT202] API Project Milestone 2 2024 M24
Student: Alanor S. (ass89)

Submissions:

Submission Selection

1 Submission [submitted] 12/11/2024 10:00:22 PM

Instructions

^ COLLAPSE ^

Implement the Milestone 2 features from the project's proposal document:
<https://docs.google.com/document/d/1XE96a8DQ52Vp49XACBDTNCq0xYDt3kF29cO88EWVwfo/view>
Make sure you add your ucid/date as code comments where code changes are done All code changes should reach the Milestone2 branch Create a pull request from Milestone2 to dev and keep it open until you get the output PDF from this assignment. Gather the evidence of feature completion based on the below tasks. Once finished, get the output PDF and copy/move it to your repository folder on your local machine. Run the necessary git add, commit, and push steps to move it to GitHub Complete the pull request that was opened earlier Create and merge a pull request from dev to prod Upload the same output PDF to Canvas

Branch name: Milestone2

Group

100%

Group: Define Appropriate Tables for Data
Tasks: 2
Points: 1

^ COLLAPSE ^

Task

100%

Group: Define Appropriate Tables for Data
Task #1: Screenshots of Table SQL
Weight: ~50%
Points: 0.50

^ COLLAPSE ^

Checklist

*The checkboxes are for your own tracking

#	Details
☐ #1	Table(s) should have the 3 core columns we'll always be using (id, created, modified) plus additional columns for the incoming API data
☐ #2	Columns should be logical and thought out (not valid to have a single field of JSON data or similar)

Sub-Task

Group: Define Appropriate Tables for Data

Task #1: Screenshots of Table SQL

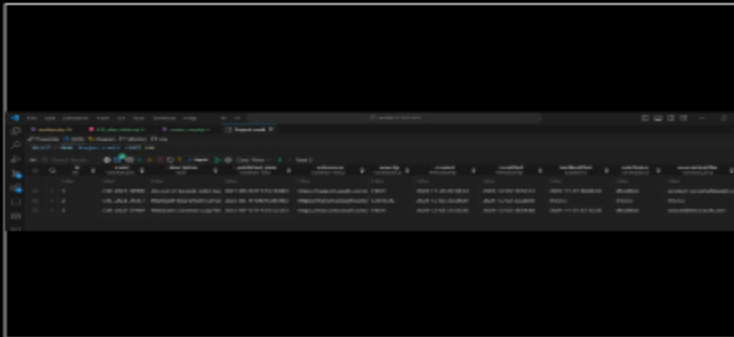
Sub Task #1: Screenshot of the table (you can duplicate this subtask as needed)

100%

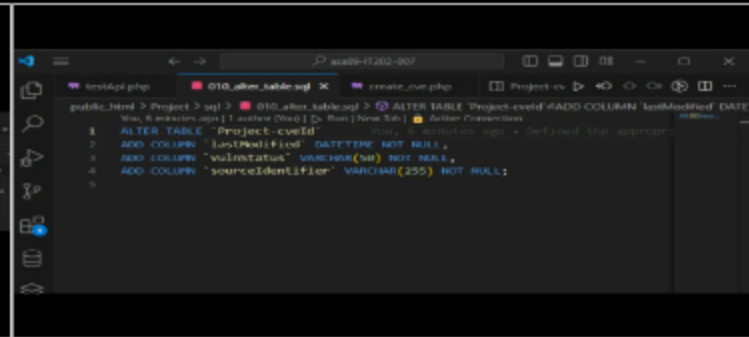
Task Screenshots

Gallery Style: 2 Columns

4 2 1



Screenshot of the table



had to update my table to include other information from the API

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Explain the columns and what data they represent (briefly), also note any normalization that may have been necessary

Response:

The id→ is an integer representing a unique identifier for each record in the table cveId → is a string representing the CVE ID. This is a unique identifier for a specific vulnerability description→ Text describing the vulnerability, including the affected system and the nature of the issue published_date→ The date and time the CVE was first published references→ A string of URLs or links to resources providing further information about the vulnerability severity→ The level of severity of the vulnerability, such as "CRITICAL", "HIGH", OR "LOW" created(automatic)→ A timestamp that tells us when the record was created in this database modified(automatic)→ The official last modification date of the CVE details lastModified→ A timestamp indicating when the record was last modified in this database vulnStatus→ CVE's status in the NVD sourceIdentifier→ The identifier of the entity that reported or provided the CVE information

NOTE: I had to alter the table because I wanted to add more columns to Project-cveId, hence the second photo

End of Task 1

Task



Group: Define Appropriate Tables for Data

Task #2: Add the pull request link for the branch related to this feature

Weight: ~50%

Points: ~0.50

^ COLLAPSE ^

i Details:

Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature.



Task URLs

URL #1

<https://github.com/AlCSS442/asa89-IT202-007/pull/48>

URL

<https://github.com/AlCSS442/asa89-IT202-007/pull/48>

End of Task 2

End of Group: Define Appropriate Tables for Data

Task Status: 2/2

Group



Group: Data Creation Page

Tasks: 4

Points: 2

^ COLLAPSE ^

Task



Group: Data Creation Page

Task #1: Screenshots of the creation page

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

i Details:

Heroku dev url must be visible in all relevant screenshots



Columns: 1

Sub-Task

Group: Data Creation Page

100%

Task #1: Screenshots of the creation page
Sub Task #1: Show potentially valid data filled in for the custom creation page

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Show potentially valid data filled in for the custom creation page

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Data Creation Page

100%

Task #1: Screenshots of the creation page

Sub Task #2: Show how the API data is fetched for API data (must be server-side)

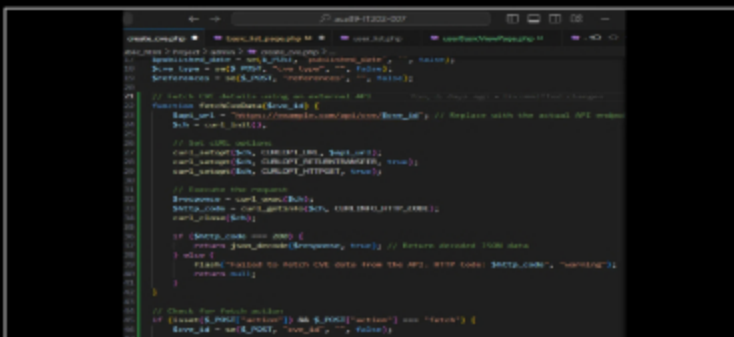
Task Screenshots

Gallery Style: 2 Columns

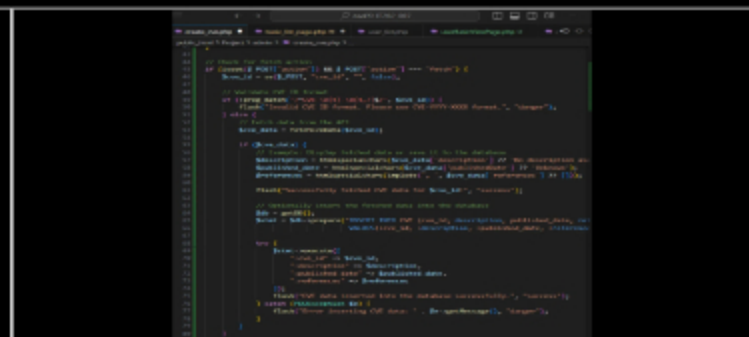
4

2

1



Show how the API data is fetched for API data (must be server-side)



Show how the API data is fetched for API data (must be server-side) part 2

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the code

Response:

I struggled with this part. When I went to office hours, I got some understanding of the logic. The gist of it was: This code allows users to manage CVE data in two ways: manually entering it or fetching it from an external API. For manual input, users can fill out a form with details like the CVE ID, description, severity, etc. The code validates the

Manual input, users can fill out a form with the CVE ID, description, severity, etc. The code validates the input to make sure the information entered is safe and valid and inserts the data into a database if everything is correct.

For fetching data, users can provide a CVE ID in a separate form. The code uses cURL to connect to an external API, retrieve the data, and display it. It also validates the CVE ID format to ensure proper input. If the user chooses, the fetched data can also be stored in the database. Throughout the process, the code uses prepared statements to prevent security issues like SQL injection and displays flash messages to guide users with success or error notifications. This makes the system both secure and user-friendly.

Sub-Task

Group: Data Creation Page

Task #1: Screenshots of the creation page

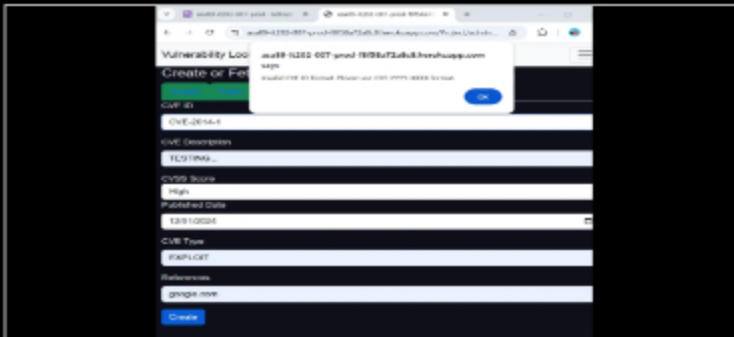
Sub Task #3: Show examples of validation messages

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1



cve id in wrong format validation

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Data Creation Page

Task #1: Screenshots of the creation page

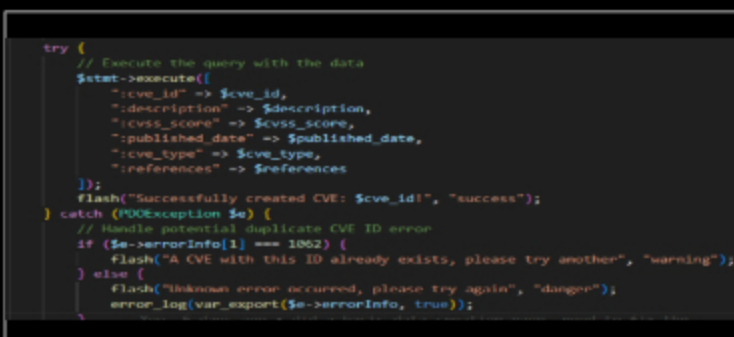
Sub Task #4: Show an example of successful creation message

100%

Task Screenshots

Gallery Style: 2 Columns

4 2 1



For some reason, I can't figure out why my message wasn't

flashing. It does let me submit the form though. I provided the code

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Data Creation Page

100%

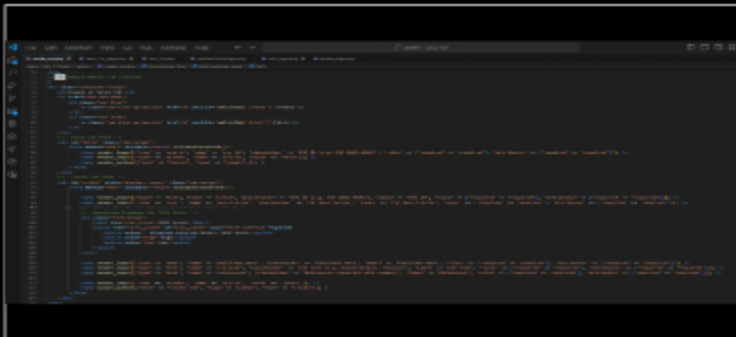
Task #1: Screenshots of the creation page

Sub Task #5: Design/Style should be considered (i.e., bootstrap, custom css, etc)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Design/Style should be considered (i.e., bootstrap, custom css, etc)

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain your design choices

Response:

The navigation tabs let users switch between the "Create" and "Fetch" CVE forms, keeping everything organized. I added form-control to the input fields, making them look consistent and easy to read. form-group helps keep the labels and fields together, making the form clean and easy to fill out. For the CVSS score, I used a dropdown to let users pick between "High" or "Low" scores, making the selection simpler and more accurate. The submit buttons are styled with btn classes, so they stand out and are easy to click. I took advantage of Bootstrap's built-in features make the page responsive, meaning it will look good on any device.

End of Task 1

Task

92%

Group: Data Creation Page

Task #2: Screenshots of creation page code

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

Details:

Include ucid/date comments for each code screenshot



Columns: 1

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #1: Form should have correct data types for each property being requested

100%

Task Screenshots

Gallery Style: 2 Columns

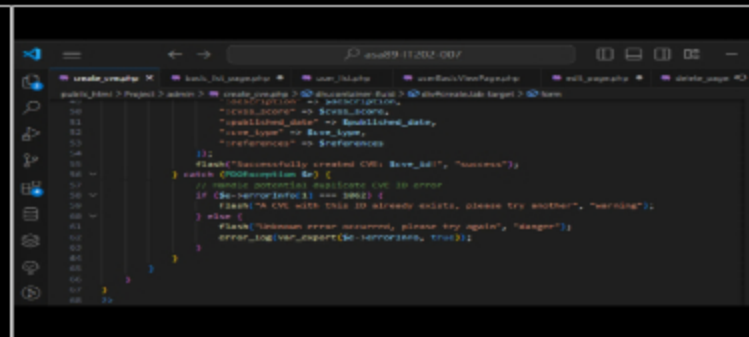
4

2

1



PART 1: Form should have correct data types for each property being requested



PART 2: Form should have correct data types for each property being requested

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly talk about each field type and why it was chosen

Response:

In the form, each field is assigned a specific data type to make sure that the correct kind of data is collected and validated:

CVE ID (type="text")→ The CVE ID needs to follow a specific format, "CVE-YYYY-XXXX", so it's validated using a regular expression. A text input type is chosen because the ID is a combination of letters, numbers, and hyphens.

Description (type="text")→ The description is a free-form text field where a user provides details about the CVE. A text input is appropriate because it allows for flexible input, though it is validated to ensure it's not left empty.

CVSS Score (type="select")→ A dropdown list is used for selecting between "High" or "Low". The selection is straightforward, and the dropdown ensures that only valid values are entered, preventing errors or invalid input.

Published Date (type="date")→ This field is used to select a date in a standard format (YYYY-MM-DD). Using the date type ensures that the date input is correctly formatted, and modern browsers will provide a date picker for convenience.

CVE Type (type="text")→ This field allows the user to input the type of CVE (e.g., Vulnerability, Exploit). Like the description, it's a text input since it's a general category that might not fit strict constraints.

References (type="text")→ This is another text input field where users provide references, usually in the form of URLs separated by commas. Validation ensures that the references are formatted correctly and can be checked against a simple URL pattern.

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #2: Form should have correct validation for each field (HTML, JS, and PHP)

100%

Task Screenshots

Gallery Style: 2 Columns

4

2

1

```
// Check if all fields are set
if (!isset($_POST["cve_id"]) && !isset($_POST["description"]) && !isset($_POST["cvss_score"]) && !isset($_POST["published_date"]) && !isset($_POST["references"])) {
    alert("All fields are required.");
    return false;
}

// Validate that the CVE ID format is correct
if (!preg_match("/^CVE-\d{4}-\d{4,7}$/", $_POST["cve_id"])) {
    alert("Invalid CVE ID format. Please use CVE-YYYY-XXXX format.");
    return false;
}
```

```
// validate cve id format
const cvePattern = /^CVE-\d{4}-\d{4,7}$/;
if (!cvePattern.test(cveId)) {
    alert("Invalid CVE ID format. Please use CVE-YYYY-XXXX format." + cveId);
    return false;
}
```

php validation for cve ID

JS validation for CVE ID

```
// Validate description (non-empty)
if (!description.trim()) {
    alert("CVE description is required.");
    return false;
}
```

```
// Validate description (non-empty)
if (!description.trim()) {
    alert("CVE description is required.");
    return false;
}
```

php validation for description

JS validation for description

```
// Validate CVSS Score HIGH VS LOW
if (cvssScore !== "high" && cvssScore !== "low") {
    alert("CVSS score must be either 'high' or 'low'.");
    return false;
}
```

```
// Validate published date
if (!publishedDate) {
    alert("Published date is required.");
    return false;
}

// Check if the published date is in a valid date format
const dateObj = new Date(publishedDate);
if (isNaN(dateObj.getTime())) {
    alert("Invalid date format. Please use YYYY-MM-DD format.");
    return false;
}

// Check if the published date is in the future
const today = new Date();
if (dateObj > today) {
    alert("Published date cannot be in the future.");
    return false;
}
```

JS validation for cvss score

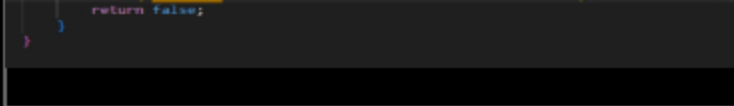
JS validation for published date

```
//validate references
if (!references.trim()) {
    alert("References are required.");
    return false;
}

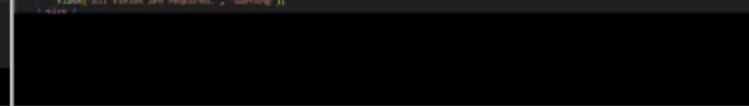
// Check if references are in a valid comma-separated format (basic validation)
const referenceList = references.split(",");
for (let ref of referenceList) {
    if (!isValidURL(ref.trim())) {
        alert("One or more references are in an invalid format.");
        return false;
    }
}
```

```
// validate references
if (!references.trim()) {
    alert("References are required.");
    return false;
}

// Check if references are in a valid comma-separated format (basic validation)
const referenceList = references.split(",");
for (let ref of referenceList) {
    if (!isValidURL(ref.trim())) {
        alert("One or more references are in an invalid format.");
        return false;
    }
}
```



JS validation for references



if empty validation for all the fields, as well as the se used to sanitize the form elements



HTML validations for all form elements(one screenshot for simplicity)

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Briefly explain the validations

Response:

1. **CVE ID: Server-Side:** In the PHP block after sanitizing the `cve_id`, the regular expression `/^CVE-\d{4}-\d{4,7}$/` is used to validate that the CVE ID matches the correct format (CVE-YYYY-XXXX). If the format is invalid, a flash message is displayed. **Client-Side:** In the `validateCreateForm()` JavaScript function, the same regular expression is used to ensure that the CVE ID follows the format before submitting the form. If invalid, it alerts the user. **HTML validation:** The field is marked with the `required` attribute, making sure that the user cannot leave it empty. Additionally, the pattern validation (regex) for the CVE ID format is implemented via JavaScript (`validateCreateForm()`).
2. **Description Server-Side:** The description field is sanitized using `htmlspecialchars()` and checked to ensure it is not empty. If it's empty, a flash message appears. **Client-Side:** In the `validateCreateForm()` JavaScript function, it checks that the description is not empty (trimmed) and alerts the user if it's missing. **HTML Validation:** The field is marked as `required`, ensuring that the user must provide a value, `required` attribute → field cannot be left empty
3. **CVSS Score Server-Side:** The `cvss_score` is sanitized and validated for correct input ("high" or "low"). If the score is not valid, a flash message is shown. **Client-Side:** In the `validateCreateForm()` function, it checks if the CVSS score is either "high" or "low" and alerts the user if the selection is invalid. **HTML Validation:** The `required` attribute on the select ensures that the user must choose a valid option (either "high" or "low"); The `required` attribute makes sure that the user must select an option from the dropdown. Also, the `disabled` attribute on the first option prevents it from being selected.
4. **Published Date:** I had trouble with this one. I should have done: **Server-Side:** The `published_date` is sanitized and can be checked later if necessary for a proper date format, though the current implementation doesn't specifically validate the date format on the server side. **Client-Side:** The commented-out code in the JavaScript validation function would check if the published date is valid (using the `Date` object), ensuring it's in the correct format (YYYY-MM-DD) and not in the future. **HTML Validation:** The `required` attribute ensures that the user cannot leave the date field empty. The `required` attribute ensures the user must select a valid date. `HTML5 type="date"` makes sure the format is recognized as a date (usually YYYY-MM-DD), but the exact format may

vary slightly depending on the browser.

5. **References: Server-Side:** The references field is sanitized, and then it checks if the field is not empty. **Client-Side:** In `validateCreateForm()`, references are validated to ensure they are not empty and are in a basic comma-separated format. Each reference is checked using a URL validation function `isValidURL()`. **HTML Validation:** The field is marked as required, ensuring the user cannot submit the form without providing references. However, the input allows the user to enter references in any format, which is later validated on the client-side with JavaScript (for comma-separated URLs). Again, the required attribute ensures that the user must provide input for references.

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #3: Successful creation should have a user-friendly message

100%

Task Screenshots

Gallery Style: 2 Columns

4

2

1

this was my code for this part

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Explain how duplicate/existing data is handled

Response:

My message didn't show up, but I was able to see that the record was inserted successfully into my database. Professor said that this was a minor issue, as long as the record was inserted correctly, it was fine. The script first collects and sanitizes the POST data from the form fields (cve_id, description, cvss_score, etc.). It checks if all required fields are provided and whether the CVE ID format is correct (using a regex pattern: `/^CVE-\d{4}-\d{4,7}$/`). After sanitizing and validating the inputs, the code proceeds to insert the data into the database using a prepared SQL statement. The critical part for handling duplicate data (in this case, duplicate CVE IDs) happens within the try-catch block, specifically in the event of a PDOException. If the database throws an error related to a duplicate entry, it is identified by the error code 1062 (which corresponds to a duplicate key error in MySQL). If a duplicate CVE ID is encountered, it triggers the exception with error code 1062. The `if ($e->errorInfo[1] === 1062)` condition checks for this specific error and shows a warning message: "A CVE with this ID already exists, please try another". If the error is not related to a duplicate, the catch block logs an unknown error. My database has a unique constraint on cve_id to avoid duplicates in the first place as well. In short, the code has validation at multiple levels: HTML/JavaScript for user input format, PHP for server-side sanitization, and database constraints to prevent duplicates.

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #4: Any errors should have user-friendly messages

100%

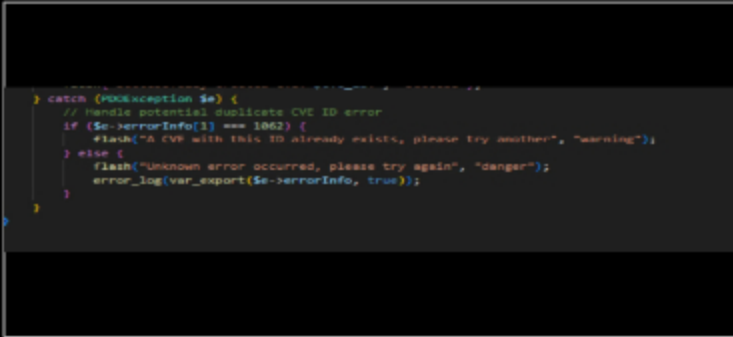
Task Screenshots

Gallery Style: 2 Columns

4

2

1



error message in code

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly describe the scenarios

Response:

My message didnt show up, but I was able to see that the record was inserted successfully into my database. This code first checks if all the required fields are filled in. If any of the fields are empty, it shows a warning message saying, "All fields are required." If all the fields are filled, it then prepares an SQL query to insert the provided data into the database. After that, it tries to execute the query. If the insertion is successful, it shows a success message. If an error occurs, such as trying to insert a duplicate CVE ID (the same ID already exists in the database), it displays a warning about the duplicate. If the error is something else, it shows a general error message and logs the error details for further investigation.

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #5: Include the form/process for fetching API data

100%

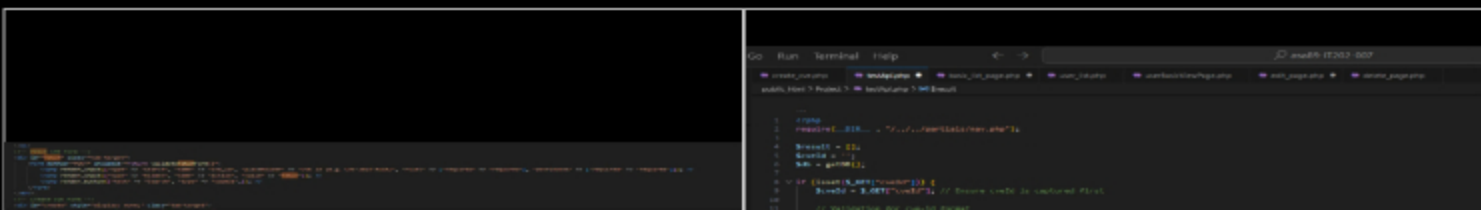
Task Screenshots

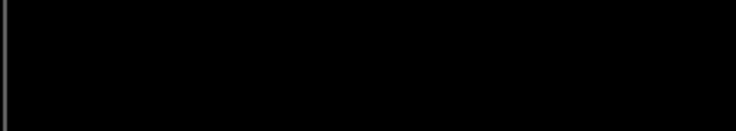
Gallery Style: 2 Columns

4

2

1





Include the form/process for fetching API data



had a helper function called `fetch_cve_data()`

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the steps (include how duplicates are handled)

Response:

The form inside the fetch div allows the user to enter a CVE ID (in the format CVE-YYYY-XXXX) to search for a specific CVE from an API.

The form uses a POST method, and it has a hidden input field to indicate the action ("fetch"), signaling that this form is for fetching data, not creating a new entry. The `validateFetchForm()` JavaScript function ensures the CVE ID is in the correct format before submitting the form. When the form is submitted, the system is expected to handle the request to fetch data from an API (though the code for the actual API call is not visible in the current code snippet). This form just handles the user input, and you would likely need to implement the backend logic to actually fetch the CVE data from the relevant API.

Sub-Task

Group: Data Creation Page

Task #2: Screenshots of creation page code

Sub Task #6: Include some indicator between custom data and API data

50%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Briefly mention what the indicator is (i.e., `api_id` if the API has `ids` or `is_api` as a boolean-like column, etc)

Response:

I had trouble with this part but after speaking with Professor, I know hwta to do. I would need to have a column (`is_api` (Boolean))→ column that is set to true (or 1) for data fetched from an API, and false (or 0) for custom data,

that way I know where the data is from I would need to output something in your HTML or JavaScript to show which data came from the form (custom data) versus the API (fetched data). To implement it, I would just insert a simple indicator when displaying the data right around here (if (isset(\$_POST['action']) && \$_POST['action'] === 'create')), I would just have to follow his example for his API

Sub-Task

100%

Group: Data Creation Page

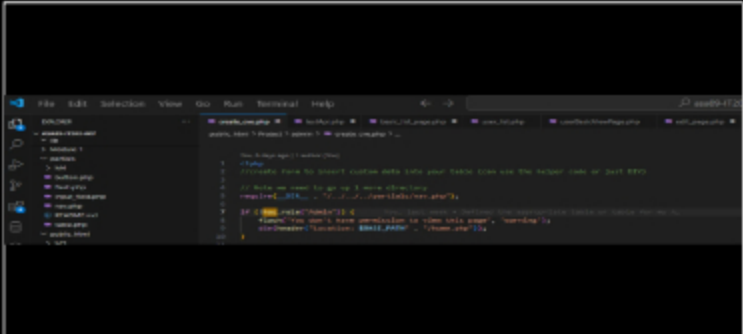
Task #2: Screenshots of creation page code

Sub Task #7: Include any other rules like role guards and login checks

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Include any other rules like role guards and login checks

Caption(s) (required) ✓
Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Briefly explain the logic/reasoning

Response:

This conditional block checks if the current user has the role of "Admin" by calling the function `has_role("Admin")`. If the user doesn't have the "Admin" role:

A flash message is set with the text "You don't have permission to view this page" and a "warning" type using the `flash()` function. The `header()` function redirects the user to the home page (`home.php`) by appending the `$BASE_PATH` variable. The `die()` function stops execution after the redirection.

End of Task 2

Task

0%

Group: Data Creation Page

Task #3: Screenshot of records from DB

Weight: ~25%

Points: ~0.50

COLLAPSE

Sub-Task

Group: Data Creation Page

Task #3: Screenshot of records from DB

Sub Task #1: Show at least one record fetched from the API

0%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown (note what differs from a custom record)*

Missing caption(s)

Sub-Task

Group: Data Creation Page

Task #3: Screenshot of records from DB

Sub Task #2: Show at least one record created via the creation form

0%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown (note what differs from the API record)*

Missing caption(s)

End of Task 3

Task

Group: Data Creation Page

100%

Task #4: Add related links
Weight: ~25%
Points: ~0.50

^ COLLAPSE ^

Checklist		*The checkboxes are for your own tracking
#	Details	
☐ #1	Include the heroku prod link for this page	
☐ #2	Add the pull request link for the branch related to this feature Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature	

Task URLs

URL #1

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/create_cve.php#

URL #2

<https://github.com/ALCSS442/asa89-IT202-007/pull/48>

End of Task 4

End of Group: Data Creation Page
Task Status: 2/4

Group

95%

Group: Data List Page (many entities)
Tasks: 3
Points: 2

^ COLLAPSE ^

Task

85%

Group: Data List Page (many entities)
Task #1: Screenshots of the list page
Weight: ~33%
Points: ~0.67

^ COLLAPSE ^

Details:

Heroku dev url must be visible in all relevant screenshots

Columns: 1

Sub-Task

Group: Data List Page (many entities)

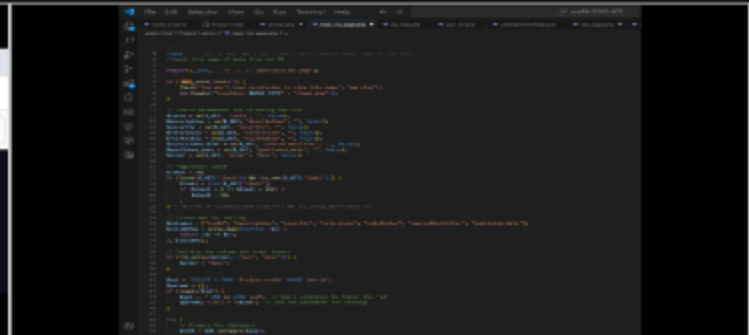
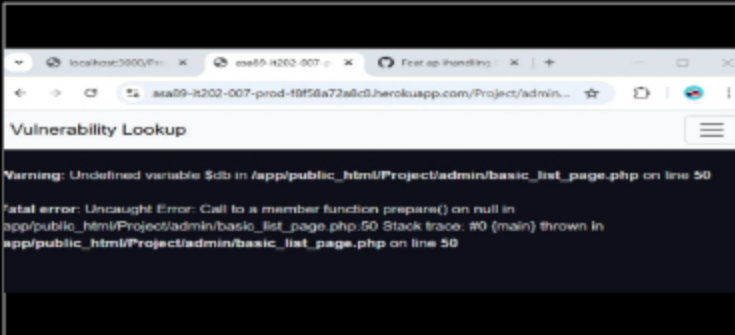
Task Screenshots

Gallery Style: 2 Columns

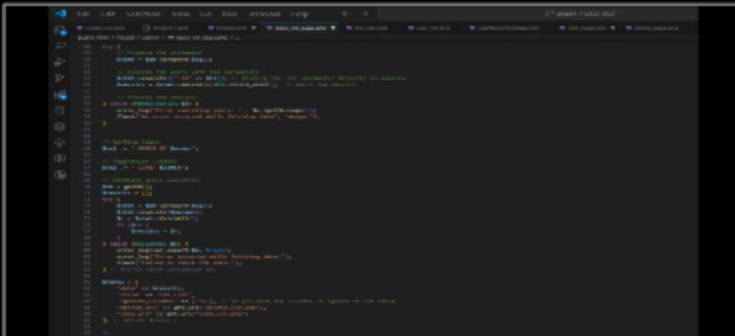
4

2

1



Show the page of your entities listed (have a reasonable number shown) code PART 1



code PART 2

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly describe the page

Response:

I had trouble with this :(I was trying to debug it to no avail, but I understood the process. Professor said to at least attach the code snippets to receive some credit. This PHP script is part of a page that allows me to view and filter a list of CVE data from a database. The first thing it does is check whether I have the "Admin" role. If I don't have the required role, it shows a warning message and redirects me to the home page. After that, the script collects search parameters from the URL, such as CVE ID, description, severity, references, and vulnerability status. It also gets the sorting order (ascending or descending) and sets a limit for how many results to show per page, which can be adjusted between 1 and 100. If no limit is specified, it defaults to 10.

The script then builds an SQL query to fetch the data from the Project-cveld table in the database, and it checks if any filters, such as CVE ID, are applied. The script uses parameterized queries to prevent SQL injection, which helps keep the application secure. If an ID filter is provided, the query includes a condition to match that ID. The script also allows me to sort the results based on the chosen order (ascending or descending again) and limits the number of records returned per page. Once the query is ready, the script executes it, retrieves the results from the database, and stores them in an array called \$results. If there's an error while fetching the data, the script catches it and logs the error, displaying a message to let me know something went wrong.

The script then decides how to display the results. If the URL includes a grid parameter, it displays the results as a grid of cards, each showing the CVE ID, description, and a "View" button to access more details about each CVE. If the grid parameter isn't set, the results are shown in a table format. To help me filter the results, there's a form at the top of the page where I can enter different search criteria. I can submit the form to apply the filters, or I can reset the form to remove them. Throughout the script, there are messages that can be displayed using the flash() function, like errors or success notifications, to keep me informed about what's happening. The script includes parts of the page from other files, such as partials/nav.php for the navigation bar and partials/flash.php for handling flash messages. In short, this script allows me to search, sort, and view CVE data in a manageable way.

Sub-Task

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #2: Show the filter/sort form based on your data and the required limit field

100%

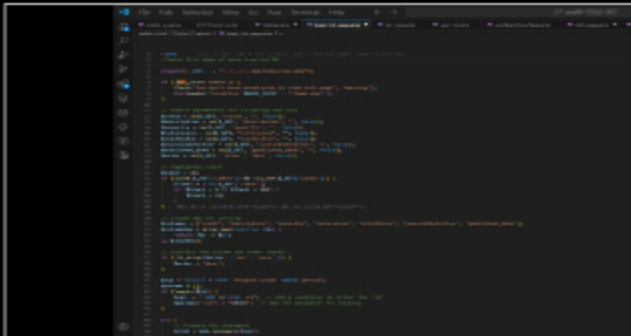
Task Screenshots

Gallery Style: 2 Columns

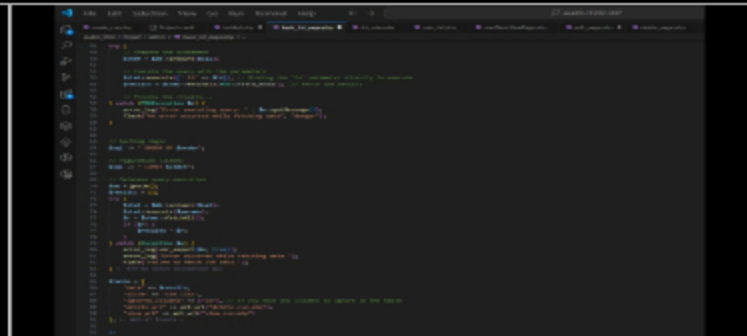
4

2

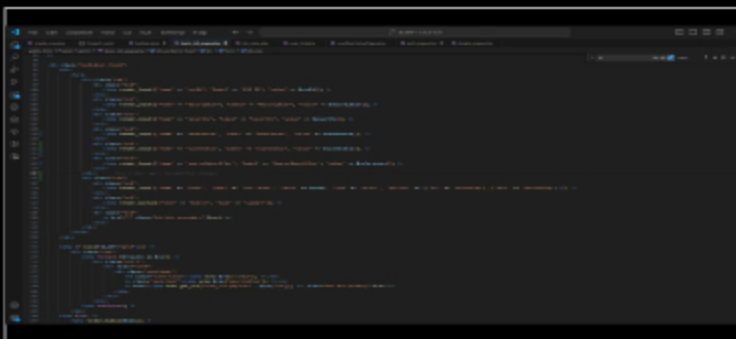
1



Show the filter/sort form based on your data and the required limit field



Show the filter/sort form based on your data and the required limit field



Show the filter/sort form based on your data and the required limit field

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Briefly mention what's available to the user

Response:

The user can filter and sort the CVE list based on the following parameters: Filters: CVE ID, Description, Severity, References, Vulnerability Status, Source Identifier. Sort Order: Choose between ascending or descending order for sorting. Search: A "Search" button to apply the filters and sort order. Reset: A "Reset" button to clear the filters and

reloading the page. The user can also adjust the number of results displayed per page through pagination (though the limit field is not shown in the form, it is part of the functionality). The user can filter the CVE list by parameters like CVE ID, description, severity, and more. They can sort the list in ascending or descending order and apply the filters using the "Search" button. There's also a "Reset" button to clear the filters. The user can control how many results are shown per page through pagination.

Sub-Task

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #3: Demonstrate a few varied filters/sorts

100%

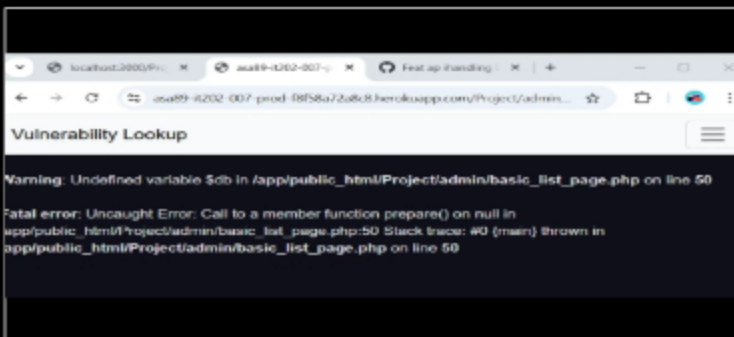
Task Screenshots

Gallery Style: 2 Columns

4

2

1



please refer to my code above

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #4: Demonstrate a filter that doesn't have any records (should show an appropriate message)

100%

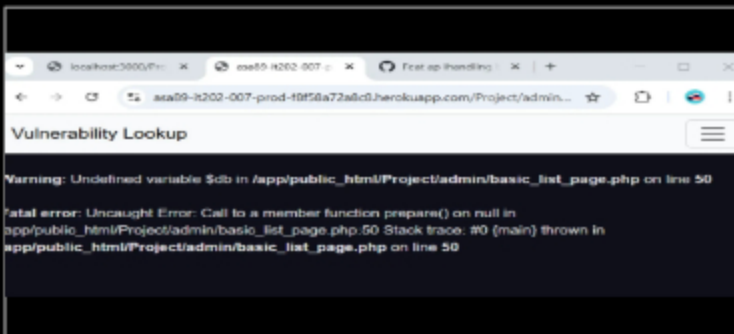
Task Screenshots

Gallery Style: 2 Columns

4

2

1



please refer to my code above

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Sub-Task

100%

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #5: Each list item should have a link of single view (i.e., details), edit, and delete (some of which may only be visible to admin users, include examples from different user roles)

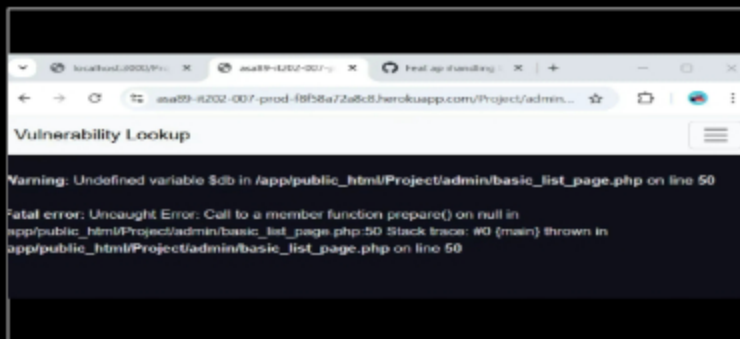
Task Screenshots

Gallery Style: 2 Columns

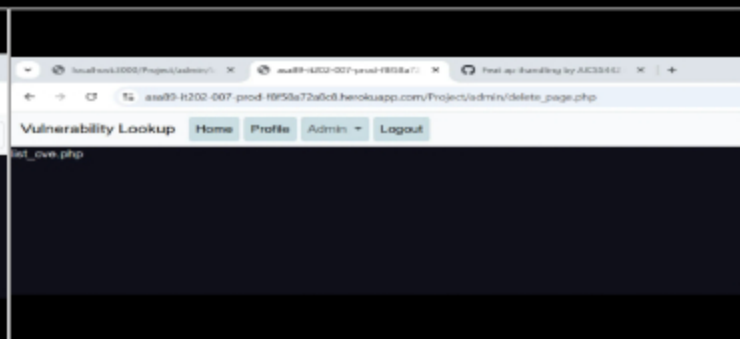
4

2

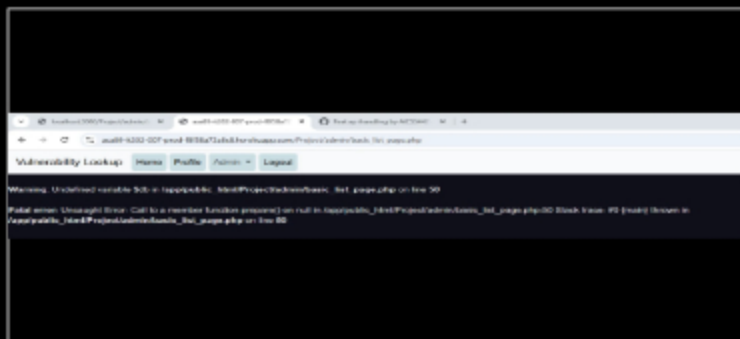
1



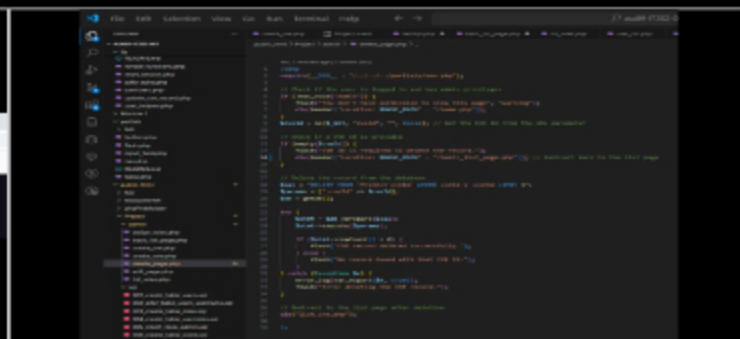
please refer to the code above



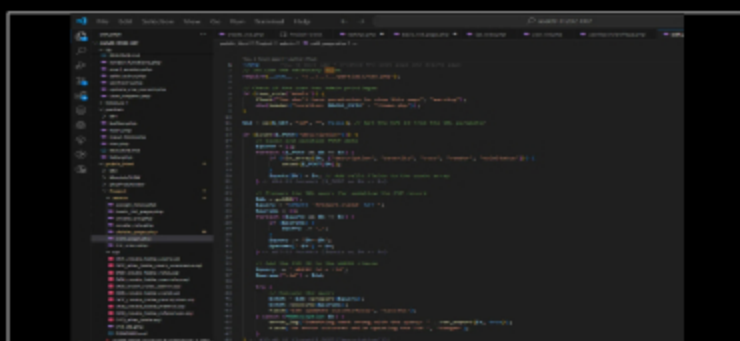
delete page, needs to be debugged



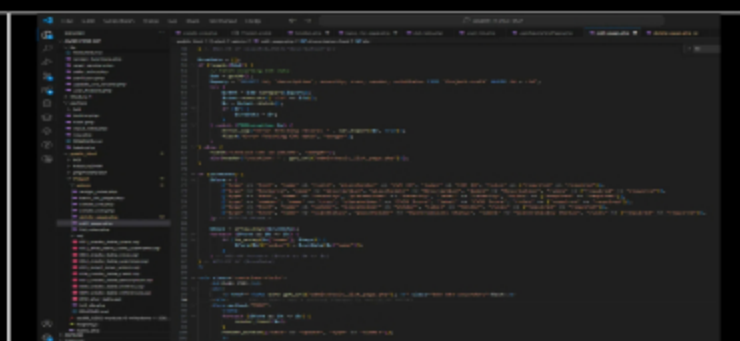
edit page, got redirected to basic_list page b/c of error, couldn't figure out how to debug this one



delete page code



edit page code part 1



edit page code part 2

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Mention which users can interact with the view, edit, and delete links

Response:

There is an admin and user list page. They are essentially the same, save for a different header. The user list has multiple entries(limit of 10 per page) and the admin one shows the record with way more detail. Only the admin has access to the edit and delete links. And any user can view.

Sub-Task

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #6: Each list item should have a summary of the entity (likely won't be the entire entity data)

0%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task

Group: Data List Page (many entities)

Task #1: Screenshots of the list page

Sub Task #7: Design/Style should be considered (i.e., bootstrap, custom css, etc)

100%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Design/Style should be considered (i.e., bootstrap, custom css, etc)

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain your design choices

Response:

For the layout, I'm using Bootstrap's grid system with row and col classes to arrange form fields and display results in an organized way. This ensures that, regardless of the device, the content will adjust and look good. I've also used Bootstrap's button styles, like btn btn-primary for the main search button, making it stand out, and btn btn-secondary for the reset button, which gives it a more subtle appearance. When displaying the CVE data, I use Bootstrap cards for each result, which are styled containers with a title, description, and a link to view more details. The results are arranged in a grid, with each card taking up a quarter of the available space on larger screens. If the grid parameter isn't set in the URL, the results are shown in a simple, styled table using Bootstrap's default table styles.

End of Task 1

Task



Group: Data List Page (many entities)
Task #2: Screenshots of the list page code
Weight: ~33%
Points: ~0.67

^ COLLAPSE ^

Details:

Include uuid/date comments for each code screenshot



Columns: 1

Sub-Task



Group: Data List Page (many entities)
Task #2: Screenshots of the list page code
Sub Task #1: Show the filter/sort form generation

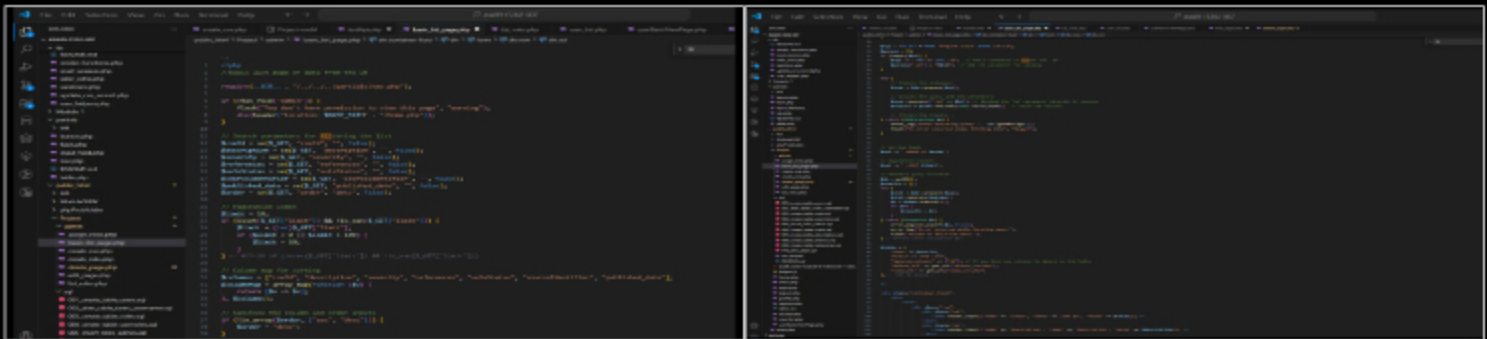
Task Screenshots

Gallery Style: 2 Columns

4

2

1



Show the filter/sort form generation

Show the filter/sort form generation





Show the filter/sort form generation

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Briefly explain how the code works (i.e, some stuff may be dynamic)

Response:

In this code, I'm building a page that fetches and displays data from a database. First, I check if the user has the "Admin" role. If not, they are redirected to another page. Then, I handle input filters from the URL using \$_GET parameters, such as CVE ID, description, severity, and others, which allow users to search for specific data. The search results are limited to 10 items per page, but this can be adjusted. The filter and sort form in the code allows users to refine and organize the displayed data. It includes input fields for filtering by CVE ID, description, severity, references, vulnerability status, and source identifier. Users can also choose the sort order from the dropdown. Once they fill out the form, they can click the "Search" button to apply the filters, which sends the selected criteria via a GET request. A "Reset" button is also available to clear the filters and refresh the page. The server then processes these inputs to filter and sort the data accordingly.

I then prepare an SQL query to fetch data from a table called Project-cveid. I apply the search filters and make sure that any user-provided inputs are sanitized. I also include sorting and pagination in the SQL query, ensuring that the data is fetched in the correct order and in manageable chunks.

The results are then displayed in a Bootstrap-based layout. If the user has requested a grid view, the results are shown in cards, each representing a CVE. Otherwise, the data is displayed in a table. There's also a form at the top to allow users to filter the data based on specific criteria like CVE ID or description. This form includes a search button and a reset link to clear the filters. If any errors occur during database queries, they are logged, and a flash message is shown to the user

Sub-Task

100%

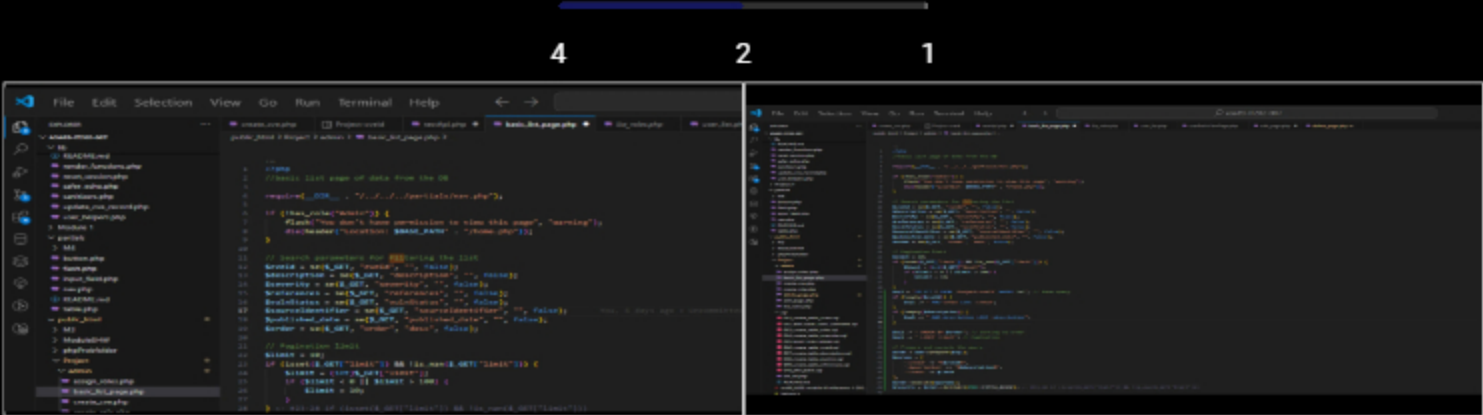
Group: Data List Page (many entities)

Task #2: Screenshots of the list page code

Sub Task #2: Show the DB query and how the filter/sort is handled (including the restriction on the limit field)

Task Screenshots

Gallery Style: 2 Columns



Show the DB query and how the filter/sort is handled

Need to fix my SQL by adding more conditions

(including the restriction on the limit field), missing SQL query here :(

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Briefly explain the related logic

Response:

My code handles filtering and sorting of the data fetched from the database based on user input. First, it gathers the search parameters (like cveld, description, severity, etc.) from the URL query string using the se() function, which sanitizes the inputs. It also handles the sort order, defaulting to descending if not specified. The pagination is handled by a limit variable, which is initially set to 10 results per page. If the limit is provided in the query string and is a valid number, it adjusts the limit accordingly, ensuring it stays between 0 and 100. I will say that my SQL query is broken. I need to adjust. The SQL query that fetches the filtered and sorted data starts with a basic SELECT * FROM Project-cveld WHERE. The query can be further modified by adding conditions based on the filters provided. For example, if a specific cveld or severity is provided, it adds AND cveld LIKE :cveld to the query. The query is ordered based on the order parameter (asc or desc) and limited by the limit variable to control pagination.

Sub-Task

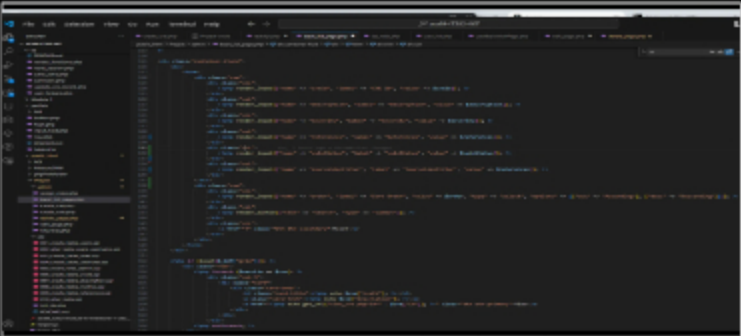
100%

Group: Data List Page (many entities)
Task #2: Screenshots of the list page code
Sub Task #3: Show how the output is generated and displayed

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1



Show how the output is generated and displayed

Caption(s) (required) ✓

Caption Hint: *Describe/highlight what's being shown*

≡ Task Response Prompt

Briefly explain the related logic

Response:

This code generates a filter form with several input fields that allow the user to filter the list of CVEs based on specific parameters. Each input field is rendered using the render_input() function, which dynamically displays fields with labels and pre-filled values based on the current search parameters. The form also includes a dropdown for sorting the results (ascending or descending) and a "Search" button to submit the form. There is also a "Reset" button to clear the filter fields and reload the page. Below the form, the results are displayed based on the user's

selection. If the grid parameter is set in the URL (isset(\$_GET["grid"])), the results are displayed in a card layout, with each CVE showing its ID and description, and a "View" button linking to the details page. If the grid parameter is not set, the results are displayed in a table format using the render_table() function.

Sub-Task

100%

Group: Data List Page (many entities)

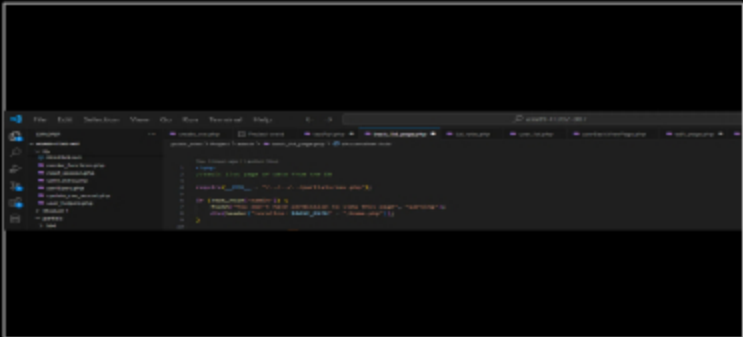
Task #2: Screenshots of the list page code

Sub Task #4: Show any restrictions like role guard or login checks

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Show any restrictions like role guard or login checks

Caption(s) (required) ✓
Caption Hint: *Describe/highlight what's being shown*

Task Response Prompt

Briefly explain the logic/reasoning

Response:

This code checks if the user has the "Admin" role using the has_role() function. If the user does not have the "Admin" role, a warning message is displayed using the flash() function, telling the user they don't have permission to view the page. Then, the header() function redirects the user to the home page, and the die() function stops further code execution, ensuring the user cannot proceed to the restricted page.

End of Task 2

Task

100%

Group: Data List Page (many entities)

Task #3: Add related links

Weight: ~33%

Points: ~0.67

^ COLLAPSE ^

Checklist

*The checkboxes are for your own tracking

#	Details
☐ #1	Include the heroku prod link for this page

#1	Include the Heroku prod link for this page
#2	Add the pull request link for the branch related to this feature Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature

Task URLs

URL #1

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/basic_list_page.php

URL

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/basic_list_page.php

URL #2

<https://github.com/AlCSS442/asa89-IT202-007/pull/52>

URL

<https://github.com/AlCSS442/asa89-IT202-007/pull/52>

End of Task 3

End of Group: Data List Page (many entities)

Task Status: 2/3

Group



Group: View Details Page (single entity)

Tasks: 3

Points: 1

^ COLLAPSE ^

Task



Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

Details:

Heroku dev url must be visible in all relevant screenshots



Columns: 1

Sub-Task



Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Sub Task #1: Entity should be fetch by id (via the url)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



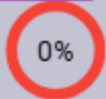
Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task



Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Sub Task #2: A missing id should redirect back to the list page with an applicable message

Task Screenshots

Gallery Style: 2 Columns

4 2 1



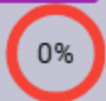
Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task



Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Sub Task #3: Design/Style should be considered (i.e., bootstrap, custom css, etc)

Task Screenshots

Gallery Style: 2 Columns

4 2 1





Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

≡ Task Response Prompt

Briefly explain your design choices

Response:

Missing Response

Sub-Task

0%

Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Sub Task #4: Data shown should be more detailed/inclusive than the summary view

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

≡ Task Response Prompt

Briefly explain the details shown and how it differs from the list view

Response:

Missing Response

Sub-Task

0%

Group: View Details Page (single entity)

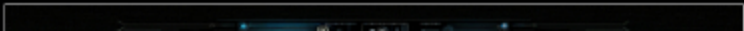
Task #1: Screenshots of the details page

Sub Task #5: There should be a link to edit the entity (this may be an admin-only thing, but it should be present for the respective role)

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1





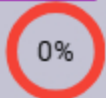
Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task



Group: View Details Page (single entity)

Task #1: Screenshots of the details page

Sub Task #6: There should be a link to delete the entity (this may be an admin-only thing, but it should be present for the respective role)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

End of Task 1

Task



Group: View Details Page (single entity)

Task #2: Screenshots of the details page code

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

Details:

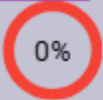
Include ucid/date comments for each code screenshot



Columns: 1

Sub-Task

Group: View Details Page (single entity)
Task #2: Screenshots of the details page code
Sub Task #1: Show how id is fetched



Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

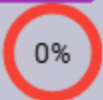
Briefly explain the logic and how it's handled when the property is missing or not "valid"

Response:

Missing Response

Sub-Task

Group: View Details Page (single entity)
Task #2: Screenshots of the details page code
Sub Task #2: Show the DB query to get the record



Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Briefly explain the logic

Response:

Missing Response

Sub-Task

Group: View Details Page (single entity)

Task #2: Screenshots of the details page code

Sub Task #3: Show the code related to presenting the data and showing the links

0%

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Briefly explain the logic

Response:

Missing Response

End of Task 2

Task

Group: View Details Page (single entity)

Task #3: Add related links

Weight: ~33%

Points: ~0.33

0%

^ COLLAPSE ^

Checklist

*The checkboxes are for your own tracking

#

Details



#1

Include the heroku prod link for this page

☐ #2

Add the pull request link for the branch related to this feature Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature

Task URLs

URL #1

Missing URL

URL

End of Task 3

End of Group: View Details Page (single entity)

Task Status: 0/3

Group



Group: Edit Data Page

Tasks: 4

Points: 2

^ COLLAPSE ^

Task



Group: Edit Data Page

Task #1: Screenshots of the edit page

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

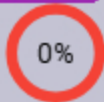
i Details:

Heroku dev url must be visible in all relevant screenshots



Columns: 1

Sub-Task



Group: Edit Data Page

Task #1: Screenshots of the edit page

Sub Task #1: Show before and after screenshots of data you'll edit (two screenshots required)

Task Screenshots

Gallery Style: 2 Columns

4

2

1





Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

≡ Task Response Prompt

Briefly explain what you changed

Response:

Missing Response

Sub-Task

0%

Group: Edit Data Page

Task #1: Screenshots of the edit page

Sub Task #2: Show examples of validation messages

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task

0%

Group: Edit Data Page

Task #1: Screenshots of the edit page

Sub Task #3: Show an example of successful edit messages

🖼 Task Screenshots

Gallery Style: 2 Columns

4 2 1





Missing Caption

Caption(s) (required)

Caption Hint: Describe/highlight what's being shown

Missing caption(s)

Sub-Task

100%

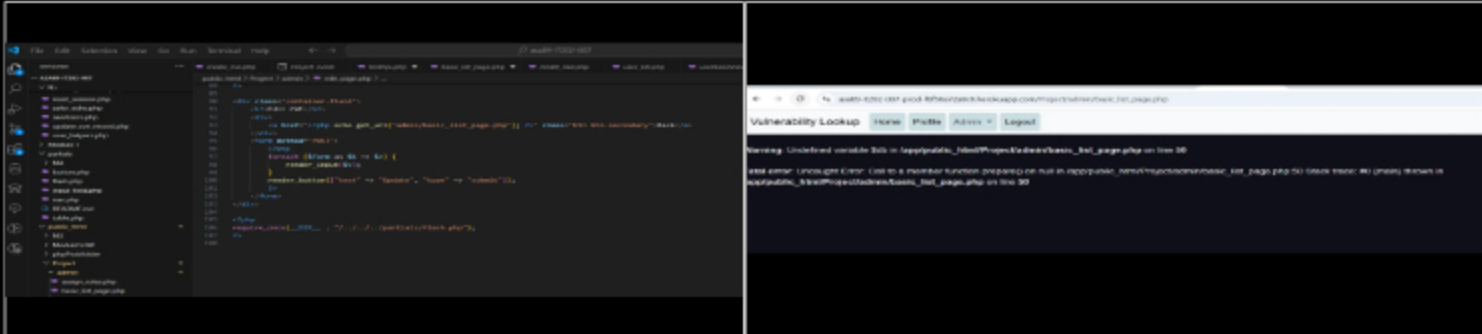
Group: Edit Data Page

Task #1: Screenshots of the edit page

Sub Task #4: Design/Style should be considered (i.e., bootstrap, custom css, etc)

Task Screenshots

Gallery Style: 2 Columns



Design/Style should be considered (i.e., bootstrap, custom css, etc) error on edit page that redirects to basic list page

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Briefly explain your design choices

Response:

It renders the necessary HTML elements, including the form fields and submit button, to provide a user-friendly interface. The code adheres to similar principles to create a well-structured and functional web application.

End of Task 1

Task

38%

Group: Edit Data Page

Task #2: Screenshots of edit page code

Weight: ~25%

Points: ~0.50

COLLAPSE

Details:

Include uuid/date comments for each code screenshot



Columns: 1

Sub-Task

Group: Edit Data Page

Task #2: Screenshots of edit page code

Sub Task #1: Form should have correct data types for each property being requested

91%

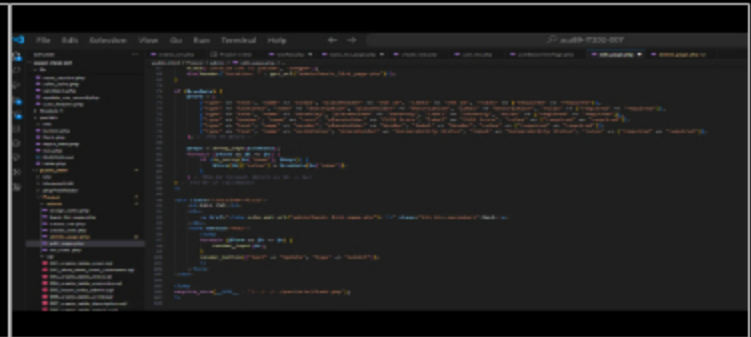
Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

part 2 of the code

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Briefly explain the data type choices

Response:

It first verifies user authorization, ensuring only authorized administrators can access the page. It then fetches existing CVE data based on the provided ID and populates a form with this information. When a user submits the form, the code sanitizes the input data to prevent security vulnerabilities and constructs an SQL query to update the database. Error handling mechanisms are in place to log errors and provide user-friendly feedback. The code renders the form with appropriate input fields and displays success or error messages to the user.

Sub-Task

Group: Edit Data Page

Task #2: Screenshots of edit page code

Sub Task #2: Form should have correct validation for each field (HTML, JS, and PHP)

0%

Task Screenshots

Gallery Style: 2 Columns

4

2

1





Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

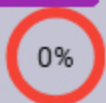
≡ Task Response Prompt

Briefly explain the validations

Response:

Missing Response

Sub-Task



Group: Edit Data Page

Task #2: Screenshots of edit page code

Sub Task #3: Successful edit should have a user-friendly message

🖼 Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

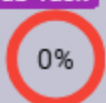
≡ Task Response Prompt

Provide a high-level step-by-step of how the fetching of the record, populating the form, and the update works and gets changed in your DB

Response:

Missing Response

Sub-Task



Group: Edit Data Page

Task #2: Screenshots of edit page code

Sub Task #4: Any errors should have user-friendly messages

🖼 Task Screenshots

4

2

1



Missing Caption

Caption(s) (required)Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt*Briefly explain the possible errors*

Response:

Missing Response

Sub-Task

100%

Group: Edit Data Page

Task #2: Screenshots of edit page code

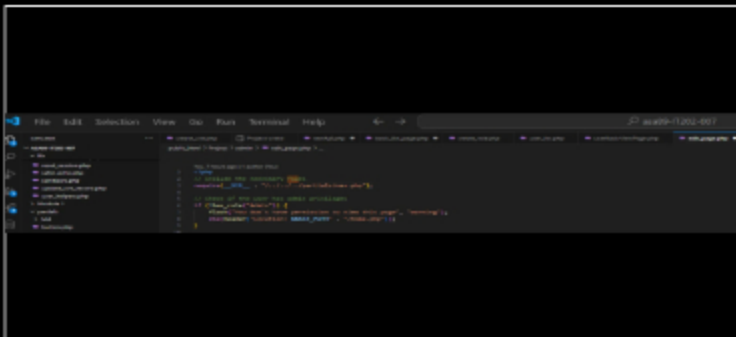
Sub Task #5: Include any other rules like role guards and login checks

Task Screenshots

4

2

1



Include any other rules like role guards and login checks

Caption(s) (required) ✓Caption Hint: *Describe/highlight what's being shown***Task Response Prompt***Briefly explain the logic/reasoning*

Response:

This conditional block checks if the current user has the role of "Admin" by calling the function `has_role("Admin")`. If the user doesn't have the "Admin" role:

A flash message is set with the text "You don't have permission to view this page" and a "warning" type using the flash() function The header() function redirects the user to the home page (home.php) by appending the \$BASE_PATH variable. The die() function stops execution after the redirection.

End of Task 2

Task



Group: Edit Data Page

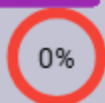
Task #3: Screenshot of records from DB

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

Sub-Task



Group: Edit Data Page

Task #3: Screenshot of records from DB

Sub Task #1: Show a before and after screenshot of the record (two screenshots)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Explain what differs

Response:

Missing Response

End of Task 3

Task



Group: Edit Data Page

Task #4: Add related links

Weight: ~25%

Points: ~0.50

^ COLLAPSE ^

Checklist

*The checkboxes are for your own tracking

#	Details
☐ #1	Include the heroku prod link for this page
☐ #2	Add the pull request link for the branch related to this feature Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature

Task URLs

URL #1

<https://github.com/AlCSS442/asa89-IT202-007/pull/52>

URL

<https://github.com/AlCSS442/asa89-IT202-007/p>

URL #2

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/edit_page.php

URL

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/edit_page.php

End of Task 4

End of Group: Edit Data Page

Task Status: 1/4

Group



Group: Delete Handling

Tasks: 3

Points: 1

^ COLLAPSE ^

Task



Group: Delete Handling

Task #1: Screenshots related to delete

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

i Details:

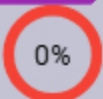
Heroku dev url must be visible in all relevant screenshots

Include ucid/date comments for each code screenshot



Columns: 1

Sub-Task



Group: Delete Handling

Task #1: Screenshots related to delete

Sub Task #1: Show the success message of a delete

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Sub-Task

Group: Delete Handling



Task #1: Screenshots related to delete

Sub Task #2: Show any error messages of a failed delete (like id not being passed)

Task Screenshots

Gallery Style: 2 Columns

4 2 1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown*

Missing caption(s)

Task Response Prompt

Explain in concise steps how this logically works

Response:

Missing Response

Sub-Task

Group: Delete Handling



Task #1: Screenshots related to delete

Sub Task #3: Show the code related to the delete processing

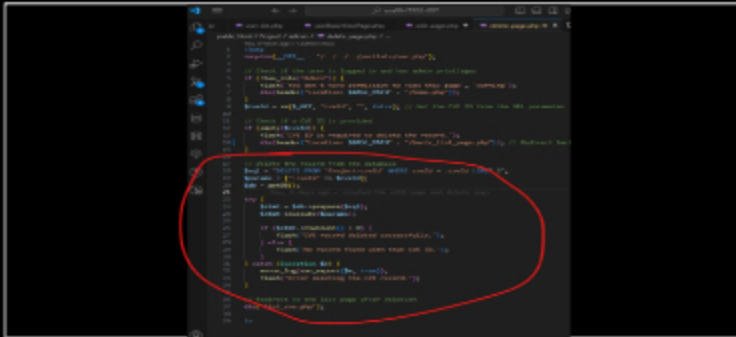
Task Screenshots

Gallery Style: 2 Columns

4

2

1



Show the code related to the delete processing

Caption(s) (required) ✓

Caption Hint: Describe/highlight what's being shown

Task Response Prompt

Explain in concise steps how this logically works

Response:

This code is used to delete a CVE record from a database. It includes a navigation file and checks if the user is logged in and has "Admin" privileges. If the user doesn't have the right permissions, it shows a warning message and redirects them to the home page. The script then retrieves the cveld from the URL. If no cveld is provided, it displays a message saying the ID is required and redirects the user back to the list page. If a valid cveld is provided, the script connects to the database and tries to delete the corresponding record. If the record is successfully deleted, it shows a success message; if no matching record is found, it informs the user. In case of any errors during the process, it logs the error and displays an error message. Finally, the script redirects the user back to the list page.

Sub-Task

Group: Delete Handling

Task #1: Screenshots related to delete

Sub Task #4: Explain the delete logic

100%

Task Response Prompt

Is it a soft or hard delete? Are there any necessary roles or restrictions? (can only delete their data, can only be done by admin, etc)?

Response:

This is a hard delete. The SQL DELETE statement permanently removes the record from the database table Project-cveld. I did not add a mechanism in the code to mark the record as deleted (soft delete), such as setting a "deleted" flag or timestamp, something to note for improvement. There is admin-only restriction however using the has_role("Admin") function in the beginning. As long as the user has admin privileges, they can delete any record.

End of Task 1

Task

Group: Delete Handling

0%

Task #2: Screenshots of the data
Weight: ~33%
Points: ~0.33

^ COLLAPSE ^

Sub-Task

Group: Delete Handling

0%

Task #2: Screenshots of the data

Sub Task #1: Show a before and after screenshot of the DB data (two screenshots)

Task Screenshots

Gallery Style: 2 Columns

4

2

1



Missing Caption

Caption(s) (required)

Caption Hint: *Describe/highlight what's being shown (note precisely what changed)*

Missing caption(s)

End of Task 2

Task

100%

Group: Delete Handling

Task #3: Add the pull request link for the branch related to this feature

Weight: ~33%

Points: ~0.33

^ COLLAPSE ^

Details:

Note: the link should end with /pull/#. Same pull request shouldn't be used for each feature



Task URLs

URL #1

<https://github.com/ALCSS442/asa89-IT202-007/pull/52>

URL

<https://github.com/ALCSS442/asa89-IT202-007/pull/52>

URL

URL #2

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/delete_page.php

https://asa89-it202-007-prod-f8f58a72a8c8.herokuapp.com/Project/admin/delete_page.php

End of Task 3

End of Group: Delete Handling

Task Status: 1/3

Group

100%

Group: Misc
Tasks: 4
Points: 1

COLLAPSE

Task

100%

Group: Misc

Task #1: Screenshot of your project board from GitHub (tasks should be in the proper column)

Weight: ~25%

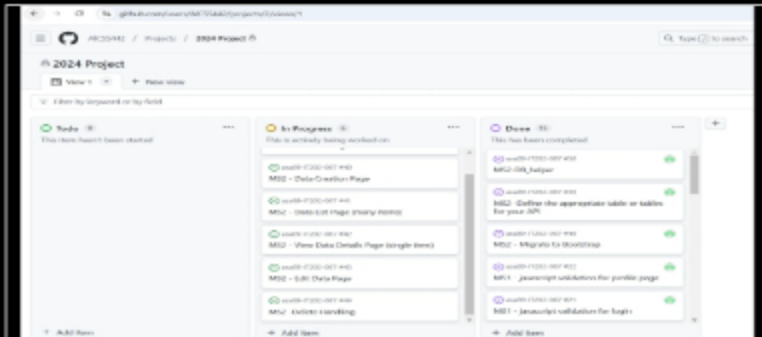
Points: ~0.25

COLLAPSE

Task Screenshots

Gallery Style: 2 Columns

4 2 1



I didnt feel confident enough to move them to done, as I was in the middle of debugging them. Milestone 1 was completely done

End of Task 1

Task

100%

Group: Misc

Task #2: Provide a direct link to the project board on GitHub

Weight: ~25%

Points: ~0.25

Task URLs

URL #1

<https://github.com/users/AlCSS442/projects/2>

URL

<https://github.com/users/AlCSS442/projects/2>

End of Task 2

Task



Group: Misc

Task #3: Talk about any issues or learnings during this assignment

Weight: ~25%

Points: ~0.25

^ COLLAPSE ^

Task Response Prompt

Response:

Just to elaborate on the Project Board. There are no Milestone 3 issues on there because honestly I was so stuck on Milestone 2, that I had a hard time starting Milestone 3. I did not close out all the issues for Milestone 2(as you can see) because I was trying so hard to debug them. I kept on getting the same error for all my admin pages "Fatal error: Uncaught Error: Call to a member function prepare() on null in /app/public_html/Project/admin/basic_list_page.php:50 Stack trace: #0 {main} thrown in /app/public_html/Project/admin/basic_list_page.php on line 50".

I had a lot of issues with Milestone 2 and onwards. I was having trouble debugging errors and understanding associative arrays for the pages. I learned a lot about the process of securely handling data deletion and the importance of role-based access control. For me, understanding the difference between hard deletes, which permanently remove records, and soft deletes, which only mark them as inactive while keeping the data for future reference or audits. I realized that hard deletes are simpler to implement, but they can lead to issues if data needs to be recovered later. I want to work on implementing soft deletes to add better admin experience. Implementing the admin-only restriction helped me see how critical it is to limit sensitive actions like deleting records to authorized users. However, I noticed that there weren't any checks to ensure users could only delete their own data, which I also need to brush up on.

I have a lot of experience with error logging now because I spent so much time trying to debug various stages of my application. Using try-catch blocks to catch exceptions, logging errors, and showing user-friendly messages taught me how to make applications more reliable and user-focused. Working with prepared statements was another important learning point. It helped me understand how they prevent SQL injection attacks by securely handling user input. I also saw how much validation is needed to sanitize user input. Honestly, I want to continue debugging and finalizing this website to make it more advanced because I see a lot of potential in it. It was very challenging, but I learned so much!

End of Task 3

Task



Group: Misc

Task #4: WakaTime Screenshot

Weight: ~25%

Points: ~0.25

^ COLLAPSE ^

Details:

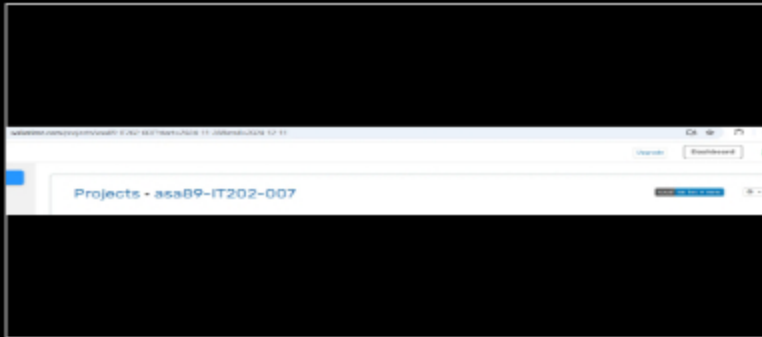
Grab a snippet showing the approximate time involved that clearly shows your repository. The duration isn't considered for grading, but there should be some time involved



Task Screenshots

Gallery Style: 2 Columns

4 2 1



WakaTime Screenshot

End of Task 4

End of Group: Misc
Task Status: 4/4

End of Assignment